Martin Nunction 1.1

Martin Nunction(a pun for Notation/Function), Is a large number notation created by Martin Wanralfs or UpBeatSquirrel925 in Fandom wiki.

A.Definitions

1.Martin Aunction (MA)

Martin Aunction is the very first function in Martin nunction, written as:

A(a,b,c)

Let a be any non-zero finite positive integer, a is the argument, a is the first entry.

Let b be any non-zero finite positive integer, b is the amount of (c-th) hyperoperation, b is the second entry.

Let c be any non-zero finite positive integer, c is the hyperoperation level, c is the third entry, where c = 1 is the Knuth up arrows Hyperoperation (↑^b, b amount)

A(a) = a

A(a,b) = a^b

General Martin Aunction expansion:

A(a,b,c) = a (c)^b a = a (c)^b-1 (a (c)^b-1 (a (c)^b-1…..#.

Right associatively, using right operand a value as the amount of iteration, a copies of a and a-1 copies of c, expanding subtract every b by 1.

(Note: octothorpe symbol means unchanged remainder of an array and hyperoperation.)

A(a,b,c) = a (c-th level hyperoperation)^b a.

General Martin Aunction leveling down:

A(a,1,c) = a (c)^1 a = a (c-1)^a a, or

A (c)^1 n = a (c-1)^n a.

Where b = 1, b value replaced by right operand a, while c is subtracted by 1, moving to (c-1)-th hyperoperation.

Examples:

A(1,b,c) = 1 (c)^b 1 = 1

A(1,1,1) = 1 ↑¹ 1 = 1¹ = 1

A(1,n,n) = 1 (n)^n 1 = 1

Where c = 1 is the Knuth's up arrow hyperoperation (↑^b, with b amount), a = 1, left a is the argument while right a is hyperoperation expansion operand, and b = 1 is the amount of current level hyperoperation, a becomes the result ignoring the second and third entry iff a = 1.

A(2,2,2) = 2 (2)² 2 = 2 (2)¹2 = 2 ↑² 2 = 2 ↑ 2 = 2² = 4.

A(2,b,c) = 2 (c)^b 2 = 2 (c)^b-1 2 = 2 (c)^b-2 2…

Operation will terminate to 4 such that a = 2.

A(3,2,1) = 3 ↑² 3 = 3 ↑ (3 ↑ 3) = 3 ↑ 27 = 7,625,597,484,987.

2.Martin Array Aunction (MAA, Level 1)

The next level in Martin Nunction, extending the array length of Martin Aunction, written as

A(a,b,c,d,e,#)

(Note: octothorpe symbol means unchanged remainder of an array and hyperoperation.)

Where array entry must be a non-zero finite positive integer or an array.

General 4 Entry Expansion

(note: do 4 entry expansion before pairing rule, such that b=d=>2):

A(ab,c,d) = A(a,b,(a,b-1,c,d),d-1)

General Pairing rule for Entry >3 where there's a number 1 followed by a number > 1, scanning left to right:

A(a,1,c,#) = A(a,(a,(...#…),c-1,#),c-1,#)

Where b is now replaced with an entire array nested into b entry c times, subtracting c by 1 on the outer array and all inner arrays.

General n entries:

A(a,b,c,d,#) → A(a,b,(a,b-1,c,d,#),d-1,#), Entries after d are for applying pairing rule such that any entry except for a = 1

Example:

A(3,2,2,2) = A(3,2,(3,1,2,2),1) → A(3,2,(3,1,2,2)), remove trailing 1, according to rule 3.

→ Applying pairing rule, for pair b=1 and c=2, (1,2).

A(3,2,(3,(3,(3,1,1,2),1,2),1,2))

→ Applying pairing rule, for pair c =1 and d = 2, (1,2).

A(3,2,(3,(3,(3,1,(3,1,(3,1,1,1),1),1),1,2),1,2))

→ Remove trailing 1s

A(3,2,(3,(3,(3,1,(3,1,(3,1,1))),1,2),1,2)).

→ Evaluate innermost array, (3,1,1) = 3 ↑ 3 = 27

A(3,2,(3,(3,(3,1,(3,1,27)),1,2),1,2)).

→ evaluate innermost array, (3,1,27), let the evaluated result called X.

A(3,2,(3,(3,(3,1,X),1,2),1,2)).

→ Evaluate innermost array, (3,1,X), let the evaluated result called X_1

A(3,2,(3,(3,X_1,1,2),1,2)).

→ Suppose we run the 4 entry Expansion for the innermost array (3,X_1,1,2) and evaluate all the inner arrays, call the evaluated result X_2

A(3,2,(3,(3,X_1,X_2,1),1,2)).

→ Remove trailing 1

A(3,2,(3,(3,X_1,X_2),1,2)).

→ Evaluate innermost array (3,X_1,X_2), call the evaluated result X_3

A(3,2,(3,X_3,1,2)).

→ Suppose we run the 4 entry Expansion for the innermost array (3,X_3,1,2) and evaluate all the inner arrays, call the evaluated result X_4

A(3,2,(3,X_3,X_4,1)).

→ Remove trailing 1

A(3,2,(3,X_3,X_4)).

→ Evaluate innermost array (3,X_3,X_4), call the evaluated result X_5

A(3,2,X_5) = 3 (X_5)² 3.

3.Martin Array Nunction (MAN, base array universe)

Martin Array Nunction(MAN)

A_(a)(n,l,s,b,t).

let (a) = array level

let n = value of entries in current array level

let l = length or amount of entries in current array level

Let s = array stack amount, separated by with |1|

Let b = current hierarchy stack level, |n|

Let t = stack hierarchy, where |n| is the first hierarchy.

A_(1)(n,4,1,1,1) = A(n,n,n,n)

A_(2)(n,3,1,1,1) = B(n,n,n)

A_(3)(n,3,2,1,1) = C(n,n,n)|1|C(n,n,n)

Each level Correspond to Alphabet tag of an array with n value, l length, and s stacks.

General Level 1 Stacking:

A_(n)(n,n,n,1,1) = n(n,n#)|1|n(n,n#)…#

n-th level array with n Length, n value and n stacks, Evaluate Rightmost Array, call the evaluated value as X.

→ #|n(n,n-1,#)|1|n(X,X,#,(X,X,#,(#)))

The second rightmost stack b Entry is reduced by 1, then the rightmost stack length and value replaced with X, then nest the entire second stack array into it's rightmost entry X times.

Example:

A_(1)(2,2,3,1,1) = A(2,2)|1|A(2,2)|1|A(2,2), calculate rightmost Array, A(2,2) = 2^2 = 4.

→ A(2,2)|1|A(2,1)|1|A(4,4,4,(4,4,4,(4,4,4,(4,4,4,(4,4,4,4))))).

→A(2,2)|1|A(2,1)|1|A(4,4,4,(4,4,4,(4,4,4,(4,4,4,(4,4,4,4))))), since b = 1, now we run the 2 entry operation, (2,1) = 2^1 = 2

→A(2,2)|1|A(1,2)|1|A(4,4#).

When b of the next array in an Array stack hits 1, we should do the pairing rule to it before continue evaluating the rightmost array, unless the entry is ≤2

Pairing rule works in Stacking and applied for every entry from b to n-th entry.

Example:

A_(1)(2,3,2,1,1) = A(2,2,2)|1|A(2,2,2), A(2,2,2) = 4.

→ A(2,1,2)|1|A(4,4,4,(4,4,4,(4,4,4,(4,4,4,(4,4,4,4)))))

→A(2,(2,(2,1,1),1),1)|1|#, (2,1,1) = 2 ↑ 2 = 4.

→A(2,(2,4,1),1), (2,4,1) = 2 ↑↑↑↑ 2 = 4.

→A(2,4,1)|1|A(4,4#), now we continue the rightmost array operation, suppose we have done the Rightmost array operation until the next array stack b entry hit 1 again but now there's no entry after it that's >1, we remove trailing 1s until we are left with 3 entries if the entry length is >3.

→A(2,1,1)|1|A(X,X#), (2,1,1) = 2 ↑ 2 = 4, we run 3 entry operation, then reduce a by 1.

→A(1,4,1)|1|#, Suppose we run the operation again until b hit 1.

→A(1,1,1)|1|A(X#), now calculate rightmost entry one more time, then remove the stack separator.

→A(X,X,X#)

3 Stack case example after the second rightmost array hit 1:

A(3,3,3)|1|A(1,1,1)|1|A(X,X#), we evaluate rightmost Array, call X_1, now we replace the second rightmost array with X_1 Entry and X_ Value too, subtracting the first array b by 1

→ A(3,2,3)|1|A(X_1,X_1#)|1|A(X_1,X_1#)

27

2 stack with 4 entry example:

A_(1)(2,4,2,1,1) = A(2,2,2,2)|1|A(2,2,2,2), Evaluate rightmost entry, A(2,2,2,2) = 4.

→A(2,1,2,2)|1|A(4,4,4,(4,4,4,(4,4,4,(4,4,4,(4,4,4,4))))), now we work on the first array, since b=1.

→A(2,(2,(2,1,1,2),1,2),1,2) #, (2,1,1,2) = 4

→A(2,(2,4,1,2),1,2) #, (2,4,1,2) =4.

→A(2,4,1,2) #, Expansion rule.

→A(2,4,(2,3,1,2),1) #, (2,3,1,2) =4.

→A(2,4,4,1) #, remove trailing 1.

→A(2,4,4) #, now we continue Evaluating the right array, suppose we have run it until b = 1 again.

→A(2,1,4) #, Pairing rule.

→A(2,(2,(2,(2,(2,1,3),3),3),3),3) #, pairing rule

→A(2,(2,(2,(2,(2,(2,(2,(2,1,2),2),2),2),3),3),3),3), pairing rule.

→A(2,(2,(2,(2,(2,(2,(2,(2,(2,(2,1,1),1),1),2),2),2),3),3),3),3), (2,1,1) = 4.

→A(2,(2,(2,(2,(2,(2,(2,(2,(2,(2,4,1),1),2),2),2),3),3),3),3),3) (2,4,1) = 4.

→A(2,(2,(2,(2,(2,(2,(2,(2,(2,4,1),2),2),2),3),3),3),3),3), every inner array collapse to 4.

→A(2,4,3) #, suppose we run rightmost array again until b =1

→A(2,1,3) #, pairing rule, since a=2, inner arrays collapse to 4.

→A(2,4,2) #, suppose we run the rightmost array again until b=1

→A(2,1,2) #, pairing rule.

→A(2,4,1) #, suppose we run the right array again until b=1.

→A(2,1,1) #, now calculate (2,1,1) = 2 ↑ 2 = 4.

→A(1,4,1) #, suppose we run the right array again until b=1

→A(1,1,1) |1| A(X,X#), we run right array one more time, call the result X_1.

→A(X_1,X_1#), X_1 value and entry, nested X_1 times in the righmost entry.

General Level 2 stacking:

A_(a)(n,n,2,2,1) = a(n,n#)|2|a(n,n#), Let the rightmost array evaluated value called X.

→ a(n,n-1,#)|2|a(n,n#)|1|…#, with X amount of arrays and X-1 amount of stack level 1.

Or

A_(a)(n,n,n,2,1) = a(n,n#)|2|a(n,n#)|2|…#|2|(a(n,n#)|2|a(n,n#)), right associatively, Evaluate Rightmost entry, Let the result called X.

→ a(n,n#)|2|…#|2|(a(n,n-1#)|2|a(n,n#)|1|…#)

Rightmost entry expand into X amount of Level 1 stacking, reducing the Second rightmost array before the rightmost |2| separator b entry by 1.

Example:

A_(1)(2,2,2,2,1) = A(2,2)|2|A(2,2), calculate rightmost Array, A(2,2) = $2^2 = 4$.

→ A(2,1)|2|A(2,2)|1|A(2,2)|1|A(2,2)|1|A(2,2).

→A(1,4)|2|A#, suppose we finished evaluating all arrays after the |2| separator, And call the Evaluated value as X.

→A(1,3)|2|A(X,X#)|1|A(X,X#)#, reduce the

Or

A_(1)(2,2,3,2,1) = A(2,2)|2|A(2,2)|2|A(2,2), calculate rightmost array Which evaluate to 4.

→ A(2,2)|2|A(2,1)|2|A(2,2)|1|A(2,2)|1|A(2,2)|1|A(2,2).

General Level n:

A_(a)(n,l,s,b,1) = a(n,n#)|n|a(n,n#)…#, evaluate Rightmost array, call X.

→ #|n|a(n,n-1#)|n-1|…#, rightmost array expand into X amount of arrays and X-1 amount of (n-1)-th stack level, subtracting the array before the rightmost |n| separator b entry by 1.

Hierarchy >1:

A_(a)(n,l,2,1,2) a(n,n#)||1||a(n,n#), calculate rightmost array, call X.

→ a(n,n-1#)|X|a(n,n,#)…#, expand to X amount of Arrays and X-1 amount of hierarchy 1 X-th Stack Level.

General n Hierarchy:

A_(a)(n,l,2,1,n) = a(n,n#)|n-th hierarchy(1)|a(n,n#), calculate righmost array, call X.

→a(n,n-1#)|n-th hierarchy(1)|a(n,n#)…#

Expand rightmost entry to X Amount of Arrays and X-1 amount of (n-1)-th hierarchy X-th Stack level, subtracting the b entry of second rightmost array before the |n-th hierarchy| separator by 1.

General Leveling down when l = 1 or 2 or 3:

A_(n)(n,1,1,1,1) = n(n) = n

→ A_(n-1)(n,n,n,n,n)

A_(n)(n,2,1,1,1) = n(n,n) = n^n, let the result be X

→ A_(n-1)(n,X,X,X,X)

A_(n)(n,3,1,1,1) = n(n,n,n) = n (n-th hyperoperation)^n n, let the result be X

→ A_(n-1)(n,X,X,X,X).

Replace parameter l and s with result of (n)-th level array calculation

Then subtract (n) by 1.

Example:

A_(2)(3,3,1,1,1) = B(3,3,3) = 3 3^3 3, call the Evaluated value X.

→ A_(1)(3,X,X,X,X)

A_(2)(3,2,1,1,1) = B(3,3) = 3^3 = 27

→ A_(1)(3,27,27,27,27) = A(3,3,3,3#)|A(3,3,3#), with 27 entries of value 3s and 27 stacks.

A_(2)(3,1,1,1,1) = B(3) = 3.

→ A_(1)(3,3,3,3,3) = A(3,3,3)|1|A(3,3,3)|1|A(3,3,3) = 3 3^3 3, call X

→ #|1|A(X,X,X,#,(X,X,#,(#))), with X entries and nested entire Array X times in the rightmost entry.

General l = 4 expansion for any (a):

A_(a)(n,4,1,1,1) = a(n,n,n,n) = a(n,n,(n,n-1,n,n),n-1)

Array with l > 4 or more than 4 entries are only for applying the pairing rule.

Example:

A_(2)(3,4,1,1,1) = B(3,3,3,3) → B(3,3,(3,2,3,3)

→B(3,3,(3,2,(3,1,3,3),2),2)

→B(3,3,(3,2,(3,(3,(3,(3,1,2,3),2,3),2,3),2,3),2),2)

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,1,1,3),1,3),2,3),2,3),2,3),2,3),2),2)

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,1,(3,1,(3,1,(3,1,1,2),2),2),3),1,3),2,3),2,3),2,3),2,3),2),2)

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,1,(3,1,(3,1,(3,1,(3,1,(3,1,1,1),1),2),2),2),3),1,3),2,3),2,3),2,3),2,3),2),2)

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,(3,1,(3,1,(3,1,(3,1,(3,1,(3,1,1)),2),2),2),3),1,3),2,3),2,3),2,3),2,3),2),2)

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,(3,1,(3,1,(3,1,(3,1,27),2),2),2),3),1,3),2,3),2,3),2,3),2,3),2),2), call innermost array evaluated value X.

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,(3,1,(3,1,(3,1,X,2),2),2),3),1,3),2,3),2,3),2,3),2,3),2),2), Suppose we run the Pairing rule of array (3,1,X,2) and evaluate the inner arrays, call the evaluted value X_1

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,(3,1,(3,1,(3,X_1,X,2),2),2),3),1,3),2,3),2,3),2,3),2,3),2),2), Suppose we run the Expansion rule of array (3,X_1,X,2) and evaluate the inner arrays, call the evaluted value X_3

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,(3,1,(3,1,(3,X_1,X_3,1),2),2),3),1,3),2,3),2,3),2,3),2,3),2),2)

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,(3,1,(3,1,(3,X_1,X_3),2),2),3),1,3),2,3),2,3),2,3),2,3),2),2), call the evaluated value of innermost array (3,X_1,X_3) as X_4.

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,(3,1,(3,1,X_4,2),2),3),1,3),2,3),2,3),2,3),2,3),2),2), Suppose we run the Pairing rule of array (3,1,X_4,2) and evaluate the inner arrays, call the evaluted value X_5

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,(3,1,(3,X_5,X_4,2),2),3),1,3),2,3),2,3),2,3),2,3)2),2), Suppose we run the Expansion rule of array (3,X_5,X_4,2) and evaluate the inner arrays, call the evaluted value X_6

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,(3,1,(3,X_5,X_6,1),2),3),1,3),2,3),2,3),2,3),2,3),2),2)

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,(3,1,(3,X_5,X_6),2),3),1,3),2,3),2,3),2,3),2,3),2),2), call the result of Innermost array (3,X_5,X_6) as X_7.

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,(3,1,X_7,2),3),1,3),2,3),2,3),2,3),2,3),2),2), call the evaluated value of innermost array (3,1,X_7,2) after pairing rule,expansion rule, and 3 entry operation as X_8

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,(3,1,X_8,3),1,3),2,3),2,3),2,3),2,3),2),2), call the evaluated value of innermost array (3,1,X_8,2) after pairing rule,expansion rule, and 3 entry operation as X_9.

→B(3,3,(3,2,(3,(3,(3,(3,(3,(3,X_9,1,3),2,3),2,3),2,3),2,3),2),2), Suppose we run the expansion rule and evaluate all inner arrays of (3,X_9,1,3), call the evaluated X_10

→B(3,3,(3,2,(3,(3,(3,(3,(3,X_9,X_10,3),2,3),2,3),2,3),2,3),2),2)

Suppose we run the Expansion rule and 3 Entry Operation for Array (3,X_9,X_10,3), call the evaluated value X_11.

→B(3,3,(3,2,(3,(3,(3,(3,X_11,2,3),2,3),2,3),2,3),2),2)

Suppose we run the Expansion rule and 3 Entry Operation for Array (3,X_11,2,3), call the evaluated value X_12.

→B(3,3,(3,2,(3,(3,(3,X_12,2,3),2,3),2,3),2),2)

Suppose we run the Expansion rule and 3 Entry Operation for Array (3,X_12,2,3), call the evaluated value X_13.

→B(3,3,(3,2,(3,(3,X_13,2,3),2,3),2),2)

Suppose we run the Expansion rule and 3 Entry Operation for Array (3,X_13,2,3), call the evaluated value X_14.

→B(3,3,(3,2,(3,(3,X_14,2,3),2,3),2),2)

Suppose we run the Expansion rule and 3 Entry Operation for Array (3,X_14,2,3), call the evaluated value X_15.

→B(3,3,(3,2,(3,X_15,2,3),2),2)

Suppose we run the Expansion rule and 3 Entry Operation for Array (3,X_15,2,3), call the evaluated value X_16.

→B(3,3,(3,2,X_16,2),2) →B(3,3,(3,2,(3,1,X_16,2),1),2),

Call the evaluated value of innermost array (3,1,X_16,2) after pairing rule,expansion rule, and 3 entry operation as X_17

→B(3,3,(3,2,X_17,1),2) → B(3,3,(3,2,X_17),2), call the Evaluated value of innermost array (3,2,X_17) as X_18

→B(3,3,X_18,2), suppose we run the expansion rule and Evaluate all inner arrays, call the Evaluated value X_19

→B(3,3,X_19,1) → B(3,3,X_19) = 3 (X_19)³ 3, call X_20.

→A_(1)(3,X_20,X_20,X_20,X_20)

General > 1 entry parameter (a):

A_(a,b)(n,l,s,b,t) = Let X = A_(a)(n,l,s,b,t)

→ A_(X,b-1)(n,l,s,b,t)

A_(a,b,#,j,k)(n,l,s,b,t) = Let X = A_(a,b,#,j)(n,l,s,b,t)

→ A_(X,X,#,X,k-1)(n,l,s,b,t)

Calculate (a) parameter with one lower entry length before k-th entry.

Then Replace every entry before k with X and subtract k-th entry by 1.

MAN Parallels:

A_[p](a)(n,l,s,b,t)

A_[1](a)(n,l,s,b,t) = A_(a)(a)(n,l,s,b,t)

Where (a) is the first parallel of MAN

Where p is the amount of parallels parameter

Example:

General 1 parallel with single entry:

A_[1](n)(n,n,n,n,n) = A_(n)(n)(n,n,n,n,n) = Let X = Result of A_(n)(n,n,n,n,n).

→ A_(n-1)(X,X,X,#)(X,X,X,X,X)

Calculate no parallel MAN, subtract first parallel first entry by 1, then replace every parameter from no parallel MAN with Calculated value and Calculated result Amount of Entries for MAN, then replace (n,l,s) parameters with calculated result.

A_[1](1)(2,2,2,1,1) = A_(1)(1)(2,2,2,1,1) = Calculate A_(1)(2,2,2,1,1) first, A_(1)(2,2,2,1,1) = A(2,2)|A(2,2), A(2,2) = 4.

→A(4,4,4,(4,4,4,(4,4,4,(4,4,4,(4,4,4,4))))), call evaluated Value X

→ A_(X,X,X,X#)(X,X,X,X,X).

(Note: A_[1](a)(n,l,s,b,t) = 1 Parallel MAN.

Parallels copy the amount of entries and value of MAN (a) parameter)

General 1 parallel with >1 entries:

A_[1](a,b,c,#,j,k)(n,l,s,b,t) = A_(a,b,c,#,j,k)(a,b,c,#,j,k)(n,l,s,b,t) = Let X = result of A_(a,b,c,#,j)(a,b,c,#,j,k)(n,l,s,b,t). → A_(X,X,X,#,X,k-1)(X,X,X,#,X,X)(X,X,X,X,X).

General n parallels with 1 entry:

A_[n](n)(n,l,s) = A_(n)(n)…#(n,l,s,b,t) = Let X = result of A_(n)…#(n,l,s,b,t).

→ A_(n-1)(X,X,#)….#(X,X,X,X,X)

Calculate [n-1] parallel MAN, subtract n-th parallel first entry by 1, replace every parallels below n-th parallels value and entry amount by calculated value, then replace (n,l,s,b,t) parameter value by calculated value

(note: Octothorpe here means unchanged remainder of parallels or arrays)

General n parallel with >1 entries:

A_[n](a,b,#,j,k)(n,l,s,b,t) = A_(a,b,#,j,k)(a,b,#,j,k)…#(n,l,s,b,t) = Let X = result of A_(a,b,#,j)(a,b,#,j,k)…#(n,l,s,b,t).

→ A_(X,X,#,X,k-1)(X,X,#,X,X)…#(X,X,X,X,X).

MAN Parallel Array with 2 entry

A_[p,q](n)(n,l,s) = Let X = Result of A_[p](n)(n,l,s).

→ A_[X,q-1](X,X,X,#)(X,X,X,X,X)

Calculate MAN Parallel array one entry lower than q, then replace every entry before q with calculated value, subtract q by 1, then replace MAN (a) parameter with Calculated value amount of entries and value, then replace (n,l,s,b,t) parameter with Calculated

General Parallel Array with n entries:

A_[p,q,#,y,z](n)(n,l,s,b,t) = Let X = result of A_[p,q,#,y](n)(n,l,s,b,t).

→ A_[X,X,#,X,z-1](X,X#)(X,X,X,X,X)

Calculate MAN Parallel array one entry lower than z-th entry, then replace every entry before z-th entry with calculated value, subtract z-th entry by 1, then replace MAN (a) parameter with Calculated value amount of entries and value, then replace (n,l,s,b,t) parameter with Calculated value.

4.Bangs (!)

An Array operator of MAN, the first priority Operator, written in the operator slots which are second to last entry in MAN p parameter entry.

A_[#,operator,z](a)(n,l,s,b,t) or A_[p,!,q](a)(n,l,s,b,t)

General entry = 2 with single or more bangs:

A_[p,!,q](a)(n,l,s,b,t) = A_[p,p,#](a)(n,l,s,b,t)

p parameter with A_[p](p)(p,p,p,p,p) entries.

General MAN a parameter array with n bangs:

A_[p,!^(n),q](a)(n,l,s,b,t) = A_[p,p,...#…,!^(n-1),a](a)(n,l,s,b,t)

MAN parallel array filled with a with A_[p](p)(p,p,p,p,p) Entries and n-1 bangs.

>2 entry with bangs recursion:

A_[p,q,!,r](a)(n,l,s,b,t) = Let X = A_[p,!,q](a)(n,l,s,b,t), calculate 2 entry p parameter with single bang first.

→A_[X,X,!,r-1](X,X,X#)(X,X,X,X,X).

A_[p,q,r,!,s](a)(n,l,s,b,t) = Let X = A_[p,q,!,r](a)(n,l,s,b,t), calculate 3 Entry p parameter with single bang first.

→ A_[X,X,X,!,s-1](X,X,X,#)(X,X,X,X,X).

General with n bangs:

A_[p,q,#,y,!^(n),z)(a)(n,l,s,b,t) = Let X = A_[p,q,#,!^(n),y](a)(n,l,s,b,t), calculate p parameter with one entry lower with n bangs first.

→A_[X,X,#,X,!^(n),z-1](X,X,X,#)(X,X,X,X,X)

Replace every entry before bang with X, along with n,l,s,b and t parameters, with X entries and values in a parameter.

5. Tiers (/n/)

The second priority operator.

A_[p,/n/,q].

General Tiering:

/n/ = Tier 1 with value n, //n// = Tier 2 with value n, (k-th tier(n) = k-th tier with value n, higher structure,tier,value = less priority.

General tier 1 with n = 1 and n > 2:

A_[p,/1/,q](a)(n,l,s,b,t) = Let X = A_[p,q](a)(n,l,s,b,t), → A_[p,!^((A_(#)),q)(a)(n,l,s,b,t)

Evaluate X, then substract tier 1 Value by 1, which becomes X amounts of Bangs, with entire function nested into bang exponent, where innermost nested function = A_[p,!^(X),q)(a)(n,l,s,b,t)

A_[p,/n/,q](a)(n,l,s,b,t) = Let X = A_[p,q](a)(n,l,s,b,t)

→A_[p,/n-1/,!^(A_(#)),q](a)(n,l,s,b,t)

Evaluate X, then substract tier 1 value by 1, which becomes X amounts of Bangs added to the second rightmost entry as higher priority operator, with entire function nested into it's exponent, Where the innermost nested function = A_[p,/n-1/,!^(X),q](a)(n,l,s,b,t)

General Tier 2 and Tier k with value n:

A_[p,//n//,q](a)(n,l,s,b,t) = Let X = A_[a,b](a)(n,l,s,b,t)

→ A_[p,//n-1//,/A_(#)/,q](a)(n,l,s,b,t)

Where innermost nested function = A_[p,//n-1//,/X/,q](a)(n,l,s,b,t)

Nest entire function X times inside Tier 1 bracket and subtract Tier 2 value by 1, then add lower tier operator on the right side of the higher tier operater in the operator slots

k-th tier:

A_[p,(k-tier(n)),q](a)(n,l,s,b,t) = let X = A_[p,q](a)(n,l,s,b,t)

→ A_[p,(k-tier(n-1)),(k-1 tier(#)),q](a)(n,l,s,b,t)

Where innermost nested function =

A_[p,(k-tier(n-1),(k-1 tier(X),q](a)(n,l,s,b,t)

Nest entire function into (k-1)-th tier bracket and subtract k-th tier value by 1, then add lower tier operator on the right side of the higher tier in the operator slots.

6.Martin Kunction (MK)

Index of Structure and tiers, written as:

M_k(m,n,l)

Let k = k-th tier structure within Martin Nunction.

Let m = tier within Current tier structure

Let n = amount of current tier within the structure

Let l = A_[l,l,#](l)(l,l,l,l,l), with l entries for p parameter.

M_k(m,n,3) = A_[3,3, (k-th structure tier m(n)), 3](3)(3,3,3,3,3)

Example:

M_1(1,1,3) = A_[3,3,/1/,3](3)(3,3,3,3,3)

M_1(2,1,3) = A_[3,3,//1//,3](3)(3,3,3,3,3)

M_2(1,2,2) = A_[2,{2},2](2)(2,2,2,2,2)

General Martin Kunction:

M_k(m,n,l) = A_[#,(k-structure tier m(n)),l](l)(l,l) = Let X = A_[#,l](l)(l,l,l,l,l)

→ A_[#,(k-structure tier m(n-1),(k-structure tier m-1(A_(#)),l](l)(l,l,l,l,l)

Calculate X, subtract k-th structure m-th tier value by 1, then add k-th Structure (m-1)-th tier with value X, nest entire function into k-th sturture (m-1)-th tier bracket.

Where Innermost nested functikn = A_[#,(k-structure m(n-1)),(k-structure tier m-1(X)),l](l)(l,l,l,l,l).

Leveling Down Structure:

A_[#,(k-structure tier 1(1)),l](l)(l,l,l) = Let X = A_[#,l](l)(l,l,l,l,l)

→ A_[#,l,(k-1 structure tier X(A_(#),l](l)(l,l,l,l,l)

Calculate X, Level down to (k-1)-th structure with tier X and Value X, nest the entire function into the (k-1)-th structure X-th tier bracket X times, with Innermost nested function = A_[#,(k-1 Structure tier X(X)),l](l)(l,l,l,l,l)

7.Martin Extended Array Nunction(MEAN)

L^v(k,n,l)

Let v = array universe

Let k = k-th structure k-th tier value k operator. Such that l > 1.

Ignore k if l = 1.

Let n = value of each entries

Let l = length of p parameter

L^1(n,n,1) = A_[n](n)(n,n,n,n,n), base Universe

L^1(n,n,2) = A_[n,(n-th structure tier n(n)),n](n)(n,n,n,n,n)

L^2(1,n,1) = A^1_[n](n)(n,n,n,n,n), array universe 1

(note:

A_ = base Universe MAN

A^n_ = n-th universe MAN

A(a,b,c#) = base Universe MA

A_n(a,b,c) = n-th universe MA)

General:

L^n(n,n,n) = A^n-1_[#,(n-th structure tier n(n)),n](n)(n,n,n,n,n)

Calculating higher universe Martin Array Auction:

A(a,b#) = Martin Array Auction, Base Universe.

A_1(a,b,#) = Martin Array Auction, Universe 1.

n(a,b,#) = MAA level n, base Universe

n_1(a,b,#) = MAA level n, universe 1.

A_1(a,b,c) = a (c-th hyperoperation level)^b a

Any v MA or MAA has the same mechanism as v = 1.

Leveling down universe:

L^2(n,n,1) = A^1_[n](n)(n,n) = Let X = result of A^1_[n](n)(n,n,n,n,n).

→ L^1(X,X,X).

B.Rules

Rule 1: MAN array or function entry must contain an array, function, or non zero positive integers, except for operator slots.

Rule 2: (#,Operators,k) or L_a(n,l,operators),b\ or (a,[n],{n},/n/,!,b) or L_a(n,l,{n},/n/,!). Higher priority operator is written right before last entry, least Priority written after Second last array entry and so on, operator slots are always second to righmost entry in MAN.

Rule 3: remove trailing 1, (#,1) → (#), or (#,j,!,1) → (#,!,j), or [#1] → [#], except for any first entry in MAN p or a parameters, except for any three entries in MA/MAA.

Rule 4: Prioritize (!) before m-th tier, then k-th structure.

Rule 5: Evaluate Innermost array/nested function first.

Rule 6: [n,/1/,/1/,n] → [n,/2/,n]

(note: Everytime we skip something to X, for example A_(a,b)(n,l,s) = Let X = result of (a)(n,l,s), it means run the entire (a)(n,l,s) until a = 1 Operation evaluated.)

Martin Wanralfs

10-10-2026